

Python Training — Day 2

Dictionary | Set | Functions

5. Dictionary

A Dictionary stores data as key-value pairs. Think of a real dictionary — you look up a word (the key) and find its meaning (the value). In Python, instead of using position numbers like lists, you use your own meaningful keys to access values.

Example — A student record:

```
student = {  
    "name"    : "Rahul",  
    "age"     : 20,  
    "city"    : "Chennai"  
}
```

Here "name", "age", "city" are the keys. "Rahul", 20, "Chennai" are the matching values.

5.1 Dictionary Powers

- **Mutable** — You can add, update, or remove key-value pairs after creation.
- **No Duplicate Keys** — Each key must be unique. If you add the same key twice, the second value replaces the first.
- **Ordered (Python 3.7+)** — Items remember the order they were inserted in.
- **Keys Must Be Immutable** — Keys can be strings, numbers, or tuples — but NOT lists.

5.2 Creating a Dictionary

Syntax:

```
my_dict = {key1: value1, key2: value2, key3: value3}
```

Examples:

```
person = {"name": "Priya", "age": 22, "city": "Bangalore"}  
ranks  = {1: "Gold", 2: "Silver", 3: "Bronze"}  
empty  = {}
```

5.3 Accessing Values

Two ways — bracket notation and get():

```
person = {"name": "Priya", "age": 22, "city": "Bangalore"}  
  
print(person["name"])      # Priya  
print(person["age"])       # 22
```

```
# get() is safer – no error if key doesn't exist
print(person.get("city"))      # Bangalore
print(person.get("phone"))     # None    (no crash)
```

Note: `person["phone"]` gives a `KeyError` if "phone" doesn't exist. `person.get("phone")` safely returns `None`. Always prefer `get()` when you are not sure if the key exists.

5.4 Adding and Updating

```
person = {"name": "Priya", "age": 22}

# Add a new key
person["city"] = "Bangalore"
print(person)    # {'name': 'Priya', 'age': 22, 'city': 'Bangalore'}

# Update an existing key
person["age"] = 23
print(person)    # {'name': 'Priya', 'age': 23, 'city': 'Bangalore'}
```

5.5 Deleting Items

```
person = {"name": "Priya", "age": 22, "city": "Bangalore"}

del person["city"]          # Delete a specific key
print(person)               # {'name': 'Priya', 'age': 22}

value = person.pop("age")    # Remove & return the value
print(value)                # 22
print(person)               # {'name': 'Priya'}
```

5.6 Dictionary Methods

```
info = {"a": 1, "b": 2, "c": 3}

info.keys()                 # dict_keys(['a', 'b', 'c'])
info.values()               # dict_values([1, 2, 3])
info.items()                # dict_items([('a', 1), ('b', 2), ('c', 3)])
info.update({"d": 4})       # Add / update multiple pairs
copy = info.copy()          # Independent copy
info.clear()                # Empty the dictionary → {}
```

5.7 Traversing a Dictionary

Method 1 — Keys only:

```
person = {"name": "Rahul", "age": 20, "city": "Chennai"}
```

```
for key in person:
    print(key)
```

Output:

```
name
age
city
```

Method 2 — Values only:

```
for value in person.values():
    print(value)
```

Output:

```
Rahul
20
Chennai
```

Method 3 — Keys AND values together (most useful):

```
for key, value in person.items():
    print(f"{key} : {value}")
```

Output:

```
name : Rahul
age : 20
city : Chennai
```

Practice — Dictionary

1. Create a dictionary for a product with keys: name, price, brand. Print each key-value pair.
2. Add a new key 'in_stock' with value True. Print the updated dictionary.
3. Update the price to a new value and print the dictionary again.
4. Ask the user to enter a student's name and marks in 3 subjects. Store in a dictionary and print.
5. Create a dictionary of 5 countries and their capitals. Print only the country names using keys().
6. Traverse a dictionary of 4 students and their scores. Print each name and score using items().

6. Set

A Set is a collection that automatically removes duplicate values and has no fixed order. Think of it as a bag — throw items in, but if you try to add the same item twice, the bag simply ignores the repeat.

Sets are especially useful when you need only unique items, or when you want to do mathematical operations like finding common elements between two groups.

6.1 Set Powers

- **Mutable** — You can add or remove items.

- **No Duplicates** — Every value must be unique. Duplicates are silently dropped.
- **Unordered** — Items have NO fixed position. You CANNOT access items by index.
- **Heterogeneous** — Can hold different types, but items must be immutable (no lists inside a set).

6.2 Creating a Set

Syntax:

```
my_set = {item1, item2, item3}
```

Examples:

```
fruits = {"apple", "banana", "cherry"}
numbers = {1, 2, 3, 4, 5}

# Duplicates are removed automatically!
nums = {1, 2, 2, 3, 3, 3}
print(nums)    # {1, 2, 3}

# Empty set - use set(), NOT {}
empty = set()    # {} would create an empty DICTIONARY
```

Note: You cannot use {} to create an empty set. Python treats {} as an empty dictionary. Always write set() for an empty set.

6.3 Set Methods

```
s = {3, 1, 4, 1, 5}    # becomes {1, 3, 4, 5}

s.add(9)                # Add one item          → {1, 3, 4, 5, 9}
s.remove(3)             # Remove 3 - ERROR if 3 not in set
s.discard(3)            # Remove 3 - NO error if not found (safer)
s.pop()                 # Remove and return a RANDOM item
s.clear()               # Empty the set
copy = s.copy()         # Independent copy
print(len(s))           # Total items in set
```

6.4 Set Operations — The Real Power of Sets

These come from mathematics. They let you compare and combine two sets very cleanly.

Setup:

```
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}
```

Union — everything from both sets (no duplicates):

```
print(A | B)            # {1, 2, 3, 4, 5, 6, 7, 8}
print(A.union(B))       # same result
```

Intersection — only items that exist in BOTH:

```
print(A & B)           # {4, 5}
print(A.intersection(B)) # same result
```

Difference — items in A but NOT in B:

```
print(A - B)           # {1, 2, 3}
print(A.difference(B)) # same result
```

Symmetric Difference — items in A or B but NOT in both:

```
print(A ^ B)           # {1, 2, 3, 6, 7, 8}
print(A.symmetric_difference(B)) # same result
```

Practice — Sets

1. Create a set of 5 numbers with duplicates. Print it and observe that duplicates are removed.
2. Add the number 100 to the set and print it.
3. Try `remove()` on a value that is not in the set. Then try `discard()`. Write down what happens.
4. Create set A of subjects you like and set B of subjects your friend likes. Find subjects both of you like.
5. Using the same sets, find all subjects that either of you likes.
6. Convert this list to a set to remove duplicates: `[1, 2, 2, 3, 4, 4, 5]`. Print the result.

7. Functions

A function is a block of code with a name. You write it once, and then you can call it (use it) as many times as you want. It avoids repeating the same code and makes your programs cleaner and easier to manage.

Think of a function like a recipe. You write the recipe once, and then you can follow it any number of times to cook the dish — with different ingredients each time if needed.

7.1 Why Use Functions?

- Write once, use many times — saves time and avoids repetition
- Makes code cleaner and easier to understand
- Easier to fix: change the function once and it is fixed everywhere
- You can pass in inputs (parameters) and get results back (return values)

7.2 Defining a Function

Syntax:

```
def function_name(parameters):
    # code block
    # ...
```

Key points:

- `def` is the keyword to define a function
- The function name should be lowercase with underscores — like `calculate_total`
- The code inside must be indented (4 spaces or 1 tab)
- To run the function, call it by name with parentheses

7.3 Simple Function — No Parameters

```
def greet():  
    print("Hello! Welcome to Python.")  
  
# Calling the function  
greet()    # Hello! Welcome to Python.  
greet()    # Call it again – same output
```

7.4 Functions with Parameters

Parameters are the inputs you pass in when calling the function. Inside the function, you use them like normal variables.

Example with one parameter:

```
def greet_user(name):  
    print(f"Hello, {name}! How are you?")  
  
greet_user("Rahul")    # Hello, Rahul! How are you?  
greet_user("Priya")    # Hello, Priya! How are you?
```

Example with multiple parameters:

```
def add(a, b):  
    print(f"Sum of {a} and {b} is {a + b}")  
  
add(10, 5)    # Sum of 10 and 5 is 15  
add(3, 7)     # Sum of 3 and 7 is 10
```

7.5 `return` — Getting a Result Back from a Function

So far our functions just print things. But often you want a function to calculate something and give you the result. That is what `return` does — it sends the result back to whoever called the function.

Example:

```
def add(a, b):  
    result = a + b  
    return result    # Send the result back  
  
total = add(10, 5)    # Store the returned value  
print(total)          # 15
```

```
print(add(3, 7) + 1)          # 11 - use the return value directly
```

Note: Once Python hits return, the function ends immediately. Any lines after return inside the same function will not run.

7.6 Default Parameters

You can set a default value for a parameter. If the caller does not provide that argument, the default is used automatically.

```
def greet(name, message="How are you?"):
    print(f"Hello, {name}! {message}")

greet("Rahul")                # Hello, Rahul! How are you?
greet("Priya", "Good morning!") # Hello, Priya! Good morning!
```

7.7 Practical Examples

Check if a number is even or odd:

```
def check_even_odd(num):
    if num % 2 == 0:
        return "Even"
    else:
        return "Odd"

print(check_even_odd(4))    # Even
print(check_even_odd(7))    # Odd
```

Find the largest number in a list:

```
def find_largest(numbers):
    largest = numbers[0]
    for num in numbers:
        if num > largest:
            largest = num
    return largest

scores = [45, 88, 72, 95, 63]
print(f"Highest score: {find_largest(scores)}")    # 95
```

Greet every student in a list:

```
def greet_students(student_list):
    for student in student_list:
        print(f"Hello, {student}!")

students = ["Arun", "Priya", "Karthik", "Meena"]
greet_students(students)
```

Output:

```
Hello, Arun!
Hello, Priya!
Hello, Karthik!
Hello, Meena!
```

Practice — Functions

1. Write a function `square(n)` that returns the square of `n`. Test it with 3, 5, and 10.
2. Write a function `is_positive(n)` that returns `True` if `n > 0`, and `False` otherwise.
3. Write a function that takes a list of numbers and returns their average.
4. Write a function `count_vowels(word)` that counts and returns the number of vowels in a word.
5. Write a function that takes a name and prints it reversed. Example: 'Rahul' → 'luhaR'
6. Write `celsius_to_fahrenheit(c)` to convert temperature. Formula: $F = (C * 9/5) + 32$
7. Write a function that takes a dictionary of student names and marks, and returns the topper's name.

Quick Reference — All Data Structures

Feature	List []	Tuple ()	Dictionary { }	Set { }
Mutable	Yes	No	Yes	Yes
Ordered	Yes	Yes	Yes (3.7+)	No
Duplicates	Yes	Yes	Keys: No, Values: Yes	No
Index access	Yes	Yes	By key only	No
Empty syntax	[]	()	{}	set()
Best for	Ordered changeable data	Fixed data	Labelled data	Unique values / set math

Day 2 — Mixed Practice Problems

These problems mix everything from both days. Take your time and think step by step.

1. Create a dictionary of 5 friends and their phone numbers. Write a function that takes the dictionary and a name, and returns the phone number. If the name is not found, return 'Not found'.
2. Ask the user to enter numbers until they type 0. Store them in a list, convert to a set to remove duplicates, sort the unique values, and print them.
3. Write a function that takes a sentence and returns a dictionary where each word is a key and its length is the value. Example: 'hi there' → {'hi': 2, 'there': 5}

4. Create set A (subjects you passed) and set B (subjects you failed). Find the union (all subjects attempted), and check whether you passed everything (failed set should be empty).
5. Write a function `build_profile(name, age, city='Unknown')` that returns a dictionary with those details. Call it twice — once with a city, once without.
6. Create a list of 10 numbers. Write a function that returns two lists — one with even numbers and one with odd numbers. Print both.